P1: Simple Calculator

AIM: Write a Python program to perform addition, subtraction, multiplication and division of two numbers.

🛒 **DAILY LIFE: Bill Calculator at a Shop**

When you buy items at a shop, the cashier adds up prices, applies discount (subtraction), and calculates GST (multiplication). This program does the same!

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print(f"Addition:       {a} + {b} = {a+b}")
print(f"Subtraction:    {a} - {b} = {a-b}")
print(f"Multiplication: {a} x {b} = {a*b}")
if b != 0:
    print(f"Division:       {a} / {b} = {a/b:.2f}")
else:
    print("Division by zero is not allowed!")
```

EXPECTED OUTPUT:

```
Enter first number: 10
Enter second number: 4
Addition:       10.0 + 4.0 = 14.0
Subtraction:    10.0 - 4.0 = 6.0
Multiplication: 10.0 x 4.0 = 40.0
Division:       10.0 / 4.0 = 2.50
```

P2: Grade Calculator

AIM: Write a Python program to input marks and display the grade using if-elif-else.

📋 **DAILY LIFE: School Report Card**

Every school gives grades based on marks. This program automates what your teacher does manually — takes marks and gives A, B, C or F grade!

```
marks = int(input("Enter your marks (0-100): "))

if marks >= 90:
    grade = "A+ (Outstanding)"
elif marks >= 75:
    grade = "A  (Excellent)"
```

```
elif marks >= 60:
    grade = "B  (Good)"
elif marks >= 40:
    grade = "C  (Average)"
else:
    grade = "F  (Fail — Please improve!)"

print(f"Marks: {marks}")
print(f"Grade: {grade}")
```

EXPECTED OUTPUT:

Enter your marks (0-100): 87

Marks: 87

Grade: A (Excellent)

P3 Multiplication Table:

AIM:

Write a program to print the multiplication table of any number using a for loop.

```
n = int(input("Enter a number: "))
print(f"\nMultiplication Table of {n}:")
print("-" * 25)
for i in range(1, 11):
    print(f"  {n} x {i:2d} = {n*i:3d}")
```

EXPECTED OUTPUT:

Enter a number: 7

Multiplication Table of 7:

---

7 x 1 = 7

7 x 2 = 14 ...up to 10

P4: Fibonacci Series

AIM: Generate Fibonacci series up to n terms. (0,1,1,2,3,5,8,13...)

🌻 **DAILY LIFE: Found in Nature!**

Fibonacci numbers appear in sunflower seeds, pine cones, and spiral shells. Each number = sum of previous two: 0+1=1, 1+1=2, 1+2=3, 2+3=5...

```
n = int(input("How many terms? "))
a, b = 0, 1
```

```
print("Fibonacci Series:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

EXPECTED OUTPUT (n=8):

Fibonacci Series: 0 1 1 2 3 5 8 13

P5: Temperature Converter

Aim: Convert temperature between Celsius and Fahrenheit. Formula: F = (C × 9/5) + 32

```
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (celsius * 9/5) + 32
kelvin = celsius + 273.15

print(f"\n{celsius}°C = {fahrenheit:.2f}°F")
print(f"{celsius}°C = {kelvin:.2f} K")
```

Enter temperature in Celsius: 37

37.0°C = 98.60°F

37.0°C = 310.15 K

LEVEL 2 — NumPy, Pandas & Matplotlib

Programs 6 to 13 • Data Analysis and Visualization

P6 Mean, Median, Mode, Std Dev

NUMPY

AIM: Calculate Mean, Median, Mode, Standard Deviation and Variance of a dataset using Python.

 **DAILY LIFE: Cricket Player Performance Analysis**

Used to analyze a batsman's consistency. Low std deviation = consistent player. High = unpredictable!

```
import numpy as np
from scipy import stats

# Runs scored in 10 matches
runs = np.array([45, 82, 23, 67, 95, 56, 38, 71, 44, 60])

mean    = np.mean(runs)
```

```
median   = np.median(runs)
mode_res = stats.mode(runs, keepdims=True)
std_dev  = np.std(runs)
variance = np.var(runs)

print("=== Cricket Performance Analysis ===")
print(f"Runs:           {runs}")
print(f"Mean (Average):   {mean:.1f}")
print(f"Median:         {median:.1f}")
print(f"Std Deviation:    {std_dev:.1f}")
print(f"Variance:       {variance:.1f}")

if std_dev < 20:
   print("Verdict: Very consistent player!")
else:
   print("Verdict: Inconsistent player.")
=== Cricket Performance Analysis ===
Runs: [45 82 23 67 95 56 38 71 44 60]
Mean (Average):   58.1
Median:         58.0
Std Deviation:    21.4
Verdict: Inconsistent player.
```

P7

Bar Graph — Subject Marks

MATPLOTLIB

AIM: Create a bar graph showing marks in different subjects for a student.

```
import matplotlib.pyplot as plt

subjects = ['AI', 'Maths', 'English', 'Physics', 'Hindi']
marks    = [92, 85, 78, 88, 80]
colors   = ['#534AB7','#1D9E75','#D85A30','#185FA5','#BA7517']

plt.figure(figsize=(8,5))
bars = plt.bar(subjects, marks, color=colors, width=0.5)
```

```
# Add value labels on each bar
for bar, mark in zip(bars, marks):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 1, str(mark),
             ha='center', fontsize=11, fontweight='bold')

plt.title("Priya's Subject-wise Marks — Class 11", fontsize=14)
plt.xlabel("Subjects")
plt.ylabel("Marks (out of 100)")
plt.ylim(0, 110)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

RESULT: A coloured bar graph showing marks for each subject with labels on top of each bar.

P8

Line Graph — Monthly Temperature

MATPLOTLIB

AIM: Plot monthly temperature data as a line graph using Matplotlib.

```
import matplotlib.pyplot as plt

months = ['Jan','Feb','Mar','Apr','May',
          'Jun','Jul','Aug','Sep','Oct','Nov','Dec']
temp   = [15,18,24,32,38,40,35,34,30,26,20,14]

plt.figure(figsize=(10,5))
plt.plot(months, temp, color='#D85A30',
         marker='o', linewidth=2, markersize=7)
plt.fill_between(months, temp, alpha=0.1, color='#D85A30')

plt.title("Monthly Temperature — Hyderabad", fontsize=14)
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```

P9

Pandas — Student DataFrame

PANDAS

AIM: Create a student DataFrame and perform operations like filtering, sorting and calculating statistics.

```
import pandas as pd

data = {
   'Name':     ['Aarav','Priya','Rahul','Sneha','Arjun','Meera'],
   'AI_Marks':  [85, 92, 73, 95, 88, 79],
   'Math_Marks':[80, 88, 65, 91, 82, 74],
   'Attendance':[92, 98, 75, 99, 88, 83]
}
df = pd.DataFrame(data)

# Add total column
df['Total'] = df['AI_Marks'] + df['Math_Marks']

print("=== Class Report ===")
print(df.to_string(index=False))
print(f"\nClass AI Average: {df['AI_Marks'].mean():.1f}")
print(f"\nTop Student: {df.loc[df['Total'].idxmax(),'Name']}")

# Filter: students with AI marks above 85
print("\nAI Toppers (>85):")
print(df[df['AI_Marks'] > 85][['Name','AI_Marks']])
```

```
=== Class Report ===
  Name  AI_Marks  Math_Marks  Attendance  Total
 Aarav        85          80          92    165
 Priya        92          88          98    180  ...
Top Student: Sneha
```

LEVEL 3 — Machine Learning Programs

Programs 14 to 20 • Linear Regression, kNN, k-Means, Scatter Plot

P10
Linear Regression — Predict Marks
ML
AIM: Use Linear Regression to predict exam marks based on study hours.
📖 DAILY LIFE: "If I study X hours, what marks will I get?"
A student wants to know — if I study 9 hours, what score can I expect? Linear Regression answers this question using past data!

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Training data: hours studied vs marks
hours = np.array([[1],[2],[3],[4],[5],[6],[7],[8],[9],[10]])
marks = np.array([35, 42, 55, 61, 68, 75, 79, 85, 90, 95])

# Train the model
model = LinearRegression()
model.fit(hours, marks)

# Predict for 7.5 hours
new_hours = [[7.5]]
predicted_marks = model.predict(new_hours)
print(f"If you study 7.5 hours → Predicted marks: {predicted_marks[0]:.1f}")

# Plot the result
plt.scatter(hours, marks, color='blue', label='Actual Data')
plt.plot(hours, model.predict(hours), color='red', label='Regression Line')
plt.title("Study Hours vs Exam Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

print(f"Slope (rate per hour): {model.coef_[0]:.2f}")
print(f"R² Score (accuracy):   {model.score(hours,marks):.3f}")
```

If you study 7.5 hours → Predicted marks: 82.3
Slope (rate per hour): 6.42
R^2 Score (accuracy): 0.991

P11
kNN — Fruit Classifier
ML
AIM: Use k-Nearest Neighbour algorithm to classify fruits based on weight and color score.
🍎 **DAILY LIFE: Sorting fruits at a fruit stall**
A fruit vendor wants to automatically sort apples (0) and oranges (1) by weight and size. kNN classifies each new fruit by looking at its 3 most similar neighbours!

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np

# Training data: [weight(g), diameter(cm)] → 0=Apple, 1=Orange
X_train = np.array([
    [182, 8.2], [167, 7.8], [176, 8.0],  # Apples
    [150, 7.0], [154, 7.2], [161, 7.5],  # Oranges
    [185, 8.3], [148, 6.9], [178, 8.1],  # Mix
    [155, 7.3]
])
y_train = np.array([0,0,0, 1,1,1, 0,1,0,1])

# Train kNN with k=3
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# Test new fruits
test_fruits = np.array([[170, 7.9], [152, 7.1]])
results = knn.predict(test_fruits)
labels = ['Apple', 'Orange']

for i, fruit in enumerate(test_fruits):
    print(f"Fruit (weight={fruit[0]}g, dia={fruit[1]}cm) → {labels[results[i]]}")
```

Fruit (weight=170g, dia=7.9cm) → Apple
Fruit (weight=152g, dia=7.1cm) → Orange

P12
k-Means — Customer Grouping
ML
AIM: Use k-Means Clustering to group customers based on age and spending score.

```
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt

# Customer data: [age, spending_score(1-100)]
customers = np.array([
   [22,75],[25,85],[23,70],[20,90],[24,80],  # Young, high spend
   [45,20],[50,30],[48,15],[52,25],[46,22],  # Middle-aged, low spend
   [35,50],[38,55],[36,48],[40,52],[37,45]   # Middle group
])

# Group into 3 customer segments
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(customers)
labels = kmeans.labels_

# Plot clusters
colors = ['#534AB7','#1D9E75','#D85A30']
segment_names = ['Young High Spenders',
                 'Middle-aged Low Spenders',
                 'Regular Customers']

plt.figure(figsize=(8,5))
for i in range(3):
   cluster = customers[labels == i]
   plt.scatter(cluster[:,0], cluster[:,1],
               color=colors[i], label=segment_names[i], s=80)
```

```
centers = kmeans.cluster_centers_
plt.scatter(centers[:,0], centers[:,1],
            color='black', marker='X', s=200, label='Centers')

plt.title("Customer Segmentation with k-Means")
plt.xlabel("Age")
plt.ylabel("Spending Score")
plt.legend(fontsize=9)
plt.grid(alpha=0.3)
plt.show()

print("Customer Groups:", labels)
```

P13

Pearson Correlation Coefficient

STATISTICS

AIM: Calculate Pearson Correlation Coefficient between study hours and marks.

📎 **WHAT IS CORRELATION?**

Correlation tells us if two things are related. +1 = perfectly related (study more → score more), -1 = opposite related, 0 = no relation.

```
import numpy as np
from scipy.stats import pearsonr

hours = np.array([2, 3, 5, 6, 7, 8, 9, 10])
marks = np.array([45, 55, 68, 72, 79, 85, 88, 94])

corr, p_value = pearsonr(hours, marks)

print(f"Pearson Correlation: {corr:.4f}")
print(f"P-value:             {p_value:.6f}")

if corr > 0.7:
   print("Interpretation: Strong Positive Correlation!")
   print("Conclusion: More study hours = Better marks ✓")
elif corr > 0.4:
   print("Interpretation: Moderate Positive Correlation")
```

```
else:
    print("Interpretation: Weak or No Correlation")
Pearson Correlation: 0.9956
P-value:            0.000002
Interpretation: Strong Positive Correlation!
Conclusion: More study hours = Better marks ✓
```